

Project Report

SpendWise - Smart Expense Tracker (Android)

Prepared For: CSE-3111 Software Development Project

Prepared By:

Name	ID	Batch
Riduan Aziz	0752220005101007	Regular-18th

Submitted to:

Md. Ishtiaq Iqbal
Assistant Professor and Chairman
Department of Computer Science and Engineering
Cox's Bazar International University

13th January 2026

Chapter 1: Introduction

1.1 Background

With the increasing cost of living, individuals need an efficient way to monitor daily expenses and manage personal finances. Traditional methods such as notebooks, spreadsheets, or mental tracking are time-consuming, inaccurate, and difficult to analyze overtime.
SpendWise is developed to provide a modern, mobile-based solution that allows users to record expenses instantly, categorize spending, analyze trends, and generate reports using a secure cloud backend.

1.2 Problem Statement

Manual expense tracking is inefficient, error-prone, and lacks real-time insights. Users face difficulties in organizing expenses, calculating monthly totals, and understanding category-wise spending patterns. There is a need for a simple, secure, and automated system to manage personal expenses effectively.

1.3 Objectives

- To allow users to add, edit, and delete expenses easily.
- To provide real-time tracking of daily, monthly, and total expenses.
- To analyze expenses category-wise using charts.
- To support offline access with automatic synchronization.
- To enable export of expense reports in CSV and PDF formats.
- To ensure secure, user-specific data storage.

1.4 Scope & Limitations

Scope

- Expense tracking for individual users.
- Firebase-based authentication and data storage.
- Category-wise and monthly expense analysis.

- Report export functionality

Limitations

- Requires internet for authentication and report upload
- Aggregation is primarily client-side
- No built-in budget alerts in the current version

Chapter 2: System Analysis & Requirements

2.1 Existing System

Currently, users track expenses using:

- Manual notebooks
- Excel spreadsheets
- Basic calculator applications

These methods lack automation, real-time analysis, security, and cloud backup.

2.2 Proposed System

SpendWise replaces manual tracking with:

- A mobile-based Android application
- Secure login using Firebase Authentication
- Real-time cloud data storage
- Automatic calculations and analytics
- Offline data persistence

2.3 Functional Requirements

- User Registration and Login
- Add, Edit, Delete Expense

- View Expense List.
- Filter expenses by date, category, and notes.
- View dashboard summary.
- Generate CSV/PDF reports..
- Profile management and password reset

2.4 Non-Functional Requirements

- **Security:** Firebase Authentication & access rules.
- **Performance:** Dashboard load < 1 second.
- **Reliability:** Crash-free usage.
- **Usability:** Simple and intuitive UI.
- **Compatibility:** Android API level 26+

2.5 Hardware & Software Requirements

Hardware

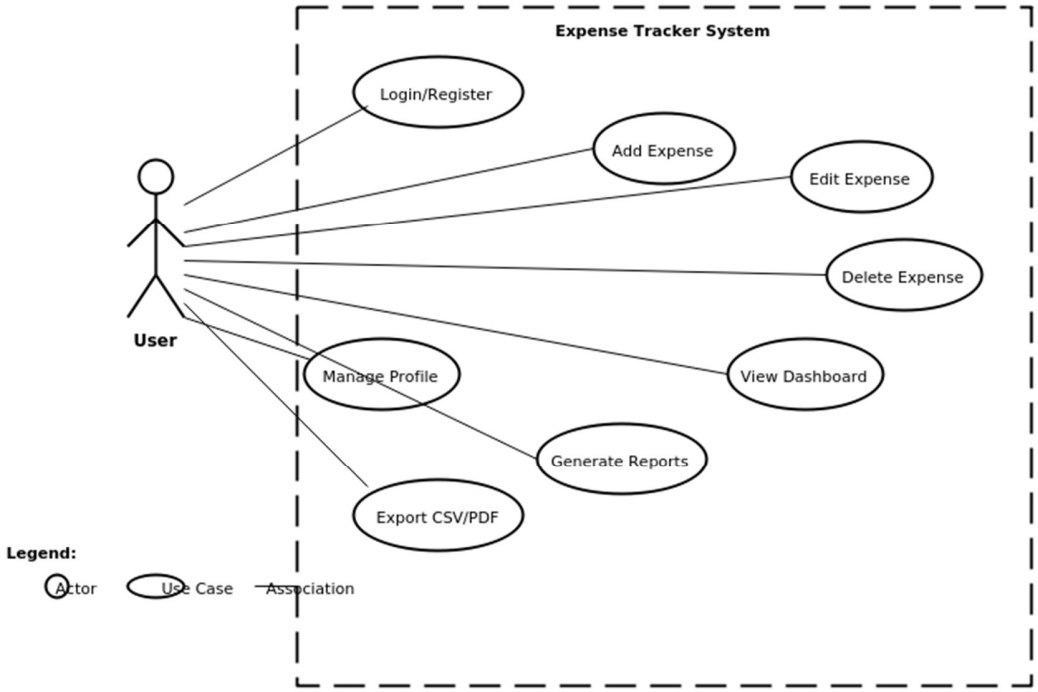
- Android smartphone
- Minimum 4GB RAM
- Internet connection

Software

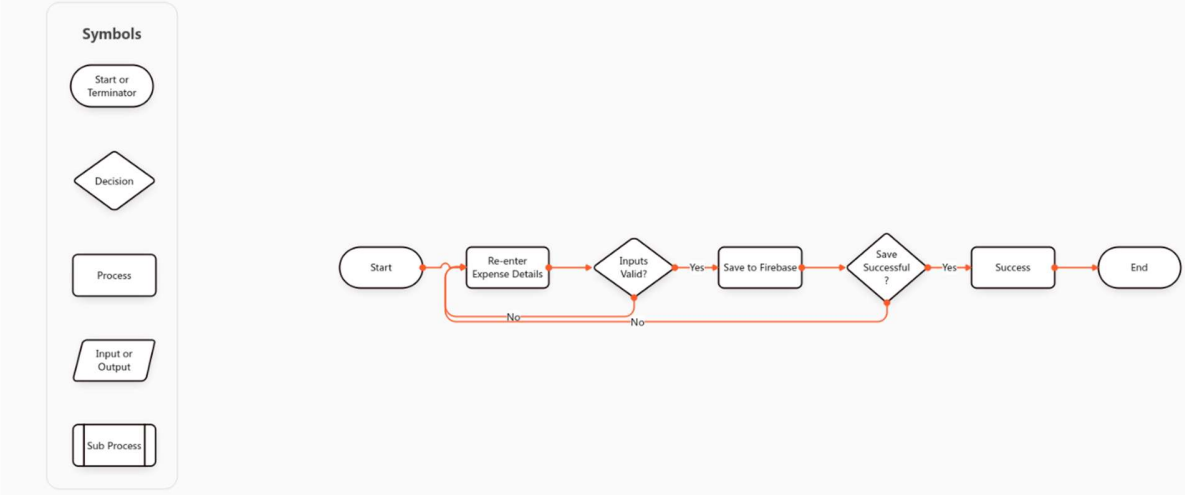
- Android Studio
- Java
- Firebase Realtime Database
- Firebase Authentication
- Firebase Storage

Chapter 3: System Design

3.1 Use Case Diagram



3.2 Activity / Flow Chart



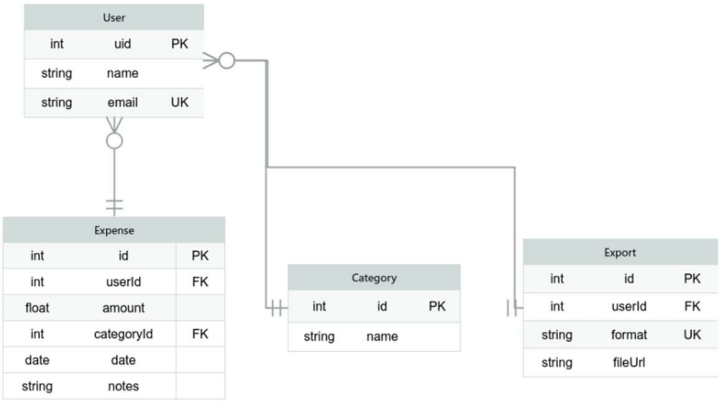
3.3 Application Package Structure

```
app/
├── manifests/
│   └── AndroidManifest.xml
│
├── java/
│   └── com/
│       └── ra/
│           └── set/
│               ├── activities/
│               │   ├── AddExpenseActivity.java
│               │   ├── DashboardActivity.java
│               │   ├── EditExpenseActivity.java
│               │   ├── ExpensesActivity.java
│               │   ├── ExportActivity.java
│               │   ├── LoginActivity.java
│               │   ├── ProfileActivity.java
│               │   ├── RegisterActivity.java
│               │   └── ReportsActivity.java
│               │
│               ├── adapters/
│               │   ├── CategorySummaryAdapter.java
│               │   └── ExpenseAdapter.java
│               │
│               ├── models/
│               │   ├── CategorySummary.java
│               │   ├── Expense.java
│               │   └── LoginRequest.java
│               │
│               ├── network/
│               │   ├── ApiClient.java
│               │   └── ApiService.java
│               │
│               └── utils/
│                   ├── DateUtils.java
│                   ├── PasswordUtils.java
│                   ├── SessionTimeout.java
│                   ├── MainActivity.java
│                   └── SETApp.java
│
├── res/
│   ├── drawable/
│   │   ├── bg_avatar.xml
│   │   ├── bg_card.xml
│   │   ├── bg_category.xml
│   │   ├── bg_circle_avatar.xml
│   │   ├── bg_icon.xml
```

```
│   │   ├── card_bg.xml
│   │   ├── ic_add.xml
│   │   ├── ic_app.xml
│   │   ├── ic_calendar.xml
│   │   ├── ic_dashboard.xml
│   │   ├── ic_expenses.xml
│   │   ├── ic_exports.xml
│   │   ├── ic_reports.xml
│   │   ├── ic_search.xml
│   │   └── ic_wallet.xml
│   │
│   ├── layout/
│   │   ├── activity_add_expense.xml
│   │   ├── activity_dashboard.xml
│   │   ├── activity_edit_expense.xml
│   │   ├── activity_expenses.xml
│   │   ├── activity_export.xml
│   │   ├── activity_login.xml
│   │   ├── activity_profile.xml
│   │   ├── activity_register.xml
│   │   ├── activity_reports.xml
│   │   ├── dialog_change_password.xml
│   │   ├── dialog_edit_profile.xml
│   │   ├── dialog_reset_password_full.xml
│   │   ├── bottom_nav.xml
│   │   ├── top_bar_card.xml
│   │   ├── layout_piechart_breakdown.xml
│   │   └── item_category_summary.xml
│   │
│   ├── menu/
│   │   └── menu_profile.xml
│   │
│   ├── values/
│   │   ├── arrays.xml
│   │   ├── colors.xml
│   │   ├── ids.xml
│   │   ├── strings.xml
│   │   └── themes.xml
│   │
│   └── xml/
│       ├── backup_rules.xml
│       └── data_extraction_rules.xml
│
├── java (generated)/
├── res (generated)/
└── Gradle Scripts/
    ├── build.gradle.kts (Project)
    ├── build.gradle.kts (Module: app)
```

```
├── gradle.properties
├── gradle-wrapper.properties
├── local.properties
├── settings.gradle.kts
├── libs.versions.toml
└── proguard-rules.pro
```

3.4 E-R Diagram



3.5 Database Schema (Firebase Realtime Database)

```
{
  "users": {
    "<authUid_1>": {
      "profile": {
        "authUid": "ND8QX0trglZjMHmVeaZoQzwXPdt1",
        "uid": "uid_1001",
        "email": "ra@ext.org",
        "name": "Riduan",
        "createdAt": 1768152531572
      },
      "expenses": {
        "-OimXs--0ix1yqm3O8by": {
          "id": "-OimXs--0ix1yqm3O8by",
          "amount": 50,
          "category": "Other",
          "note": "N",
          "date": "2026-01-12",
          "timestamp": 1768154400000
        },
```

```
      "summaries": {
        "global": {
          "total": 5690,
          "byCategory": {
            "Food": 2300,
            "Transport": 800,
            "Utilities": 600,
            "Other": 1990
          },
          "byMonth": {
            "2026-01": {
              "total": 1200,
              "Food": 500,
              "Other": 700
            },
            "2025-12": {
              "total": 1940,
              "Transport": 800,
              "Utilities": 600,
              "Other": 540
            }
          }
        }
      }
    },
  },
  "appSettings": {
    "categories": {
      "Food": true,
      "Transport": true,
      "Utilities": true,
      "Entertainment": true,
      "Other": true
    },
    "currency": "USD",
    "defaultView": "dashboard"
  },
  "meta": {
    "lastUpdated": 1768200000000,
    "version": "1.0.0"
  }
}
```

Root Structure

```
Root
|
├── users
|   ├── {authUid_1}
|   |   ├── profile
|   |   ├── expenses
```

```
│   │   ├── categories
│   │   └── summaries
│   ├── {authUid_2}
│   │   ├── profile
│   │   ├── expenses
│   │   ├── categories
│   │   └── summaries
│   └── ...
│
├── appSettings
│   ├── categories
│   ├── currency
│   └── defaultView
│
└── meta
    ├── lastUpdated
    └── version
```

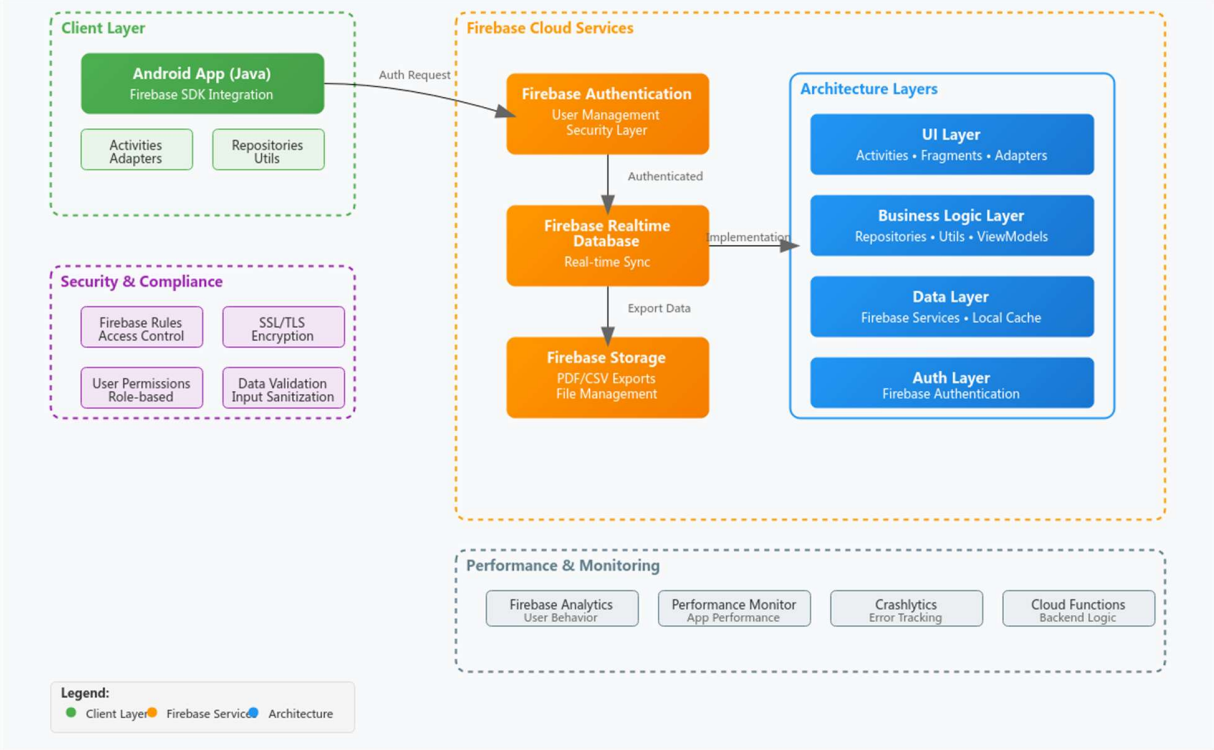
Expense Object

- userId
- expenseId
- amount
- categoryName
- notes
- date
- timestamp
- createdAt

Aggregates (Derived Data)

- All Time totals
- Daily totals
- Monthly totals
- Category totals

3.6 System Architecture

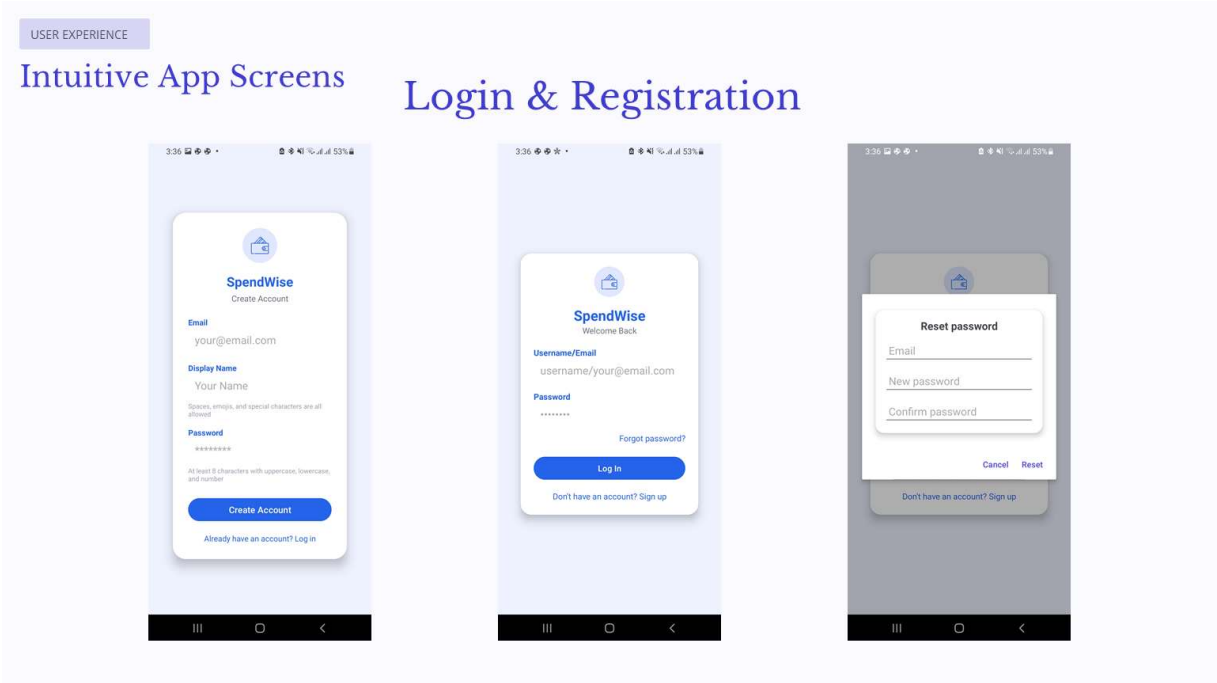


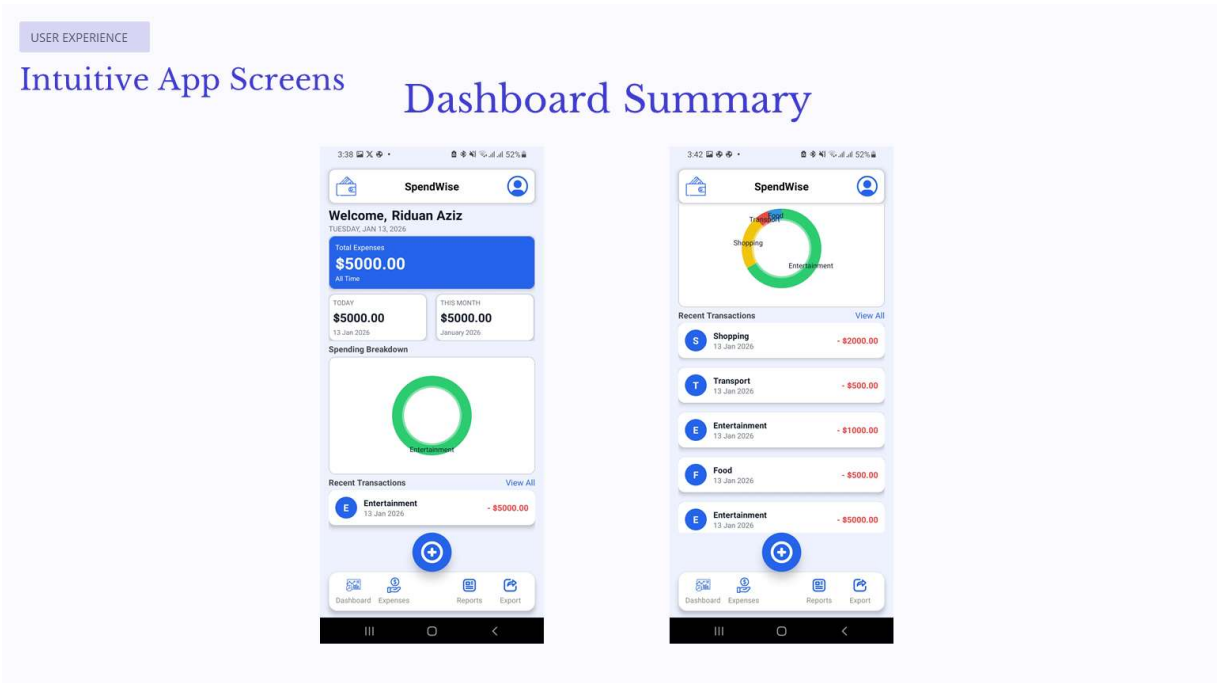
Chapter 4: Implementation

4.1 Technology Stack Description

- **Frontend:** Android (Java)
- **Backend:** Firebase Realtime Database
- **Authentication:** Firebase Authentication
- **Storage:** Firebase Storage

4.2 UI Screenshots (Captions)

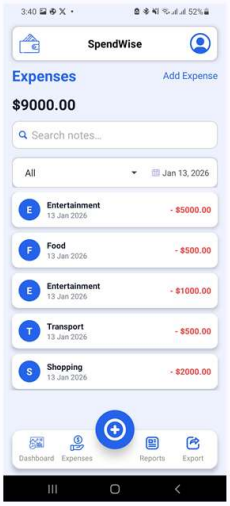


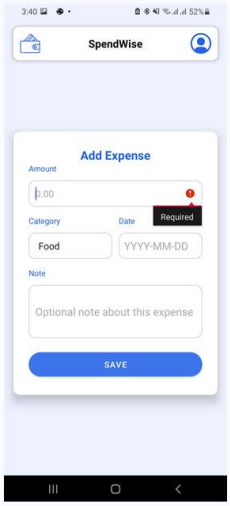


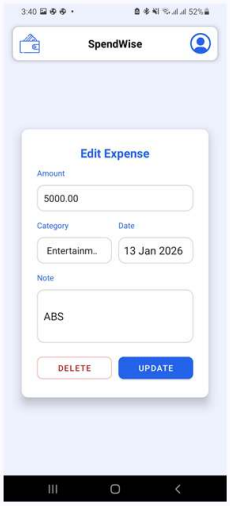
USER EXPERIENCE

Intuitive App Screens

View /Add / Edit Expense



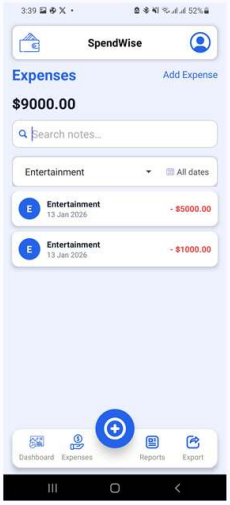


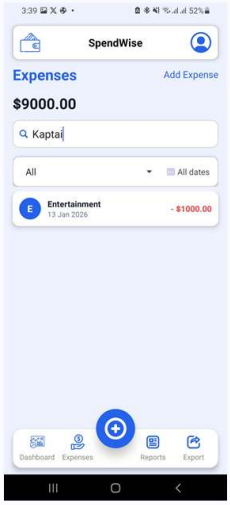


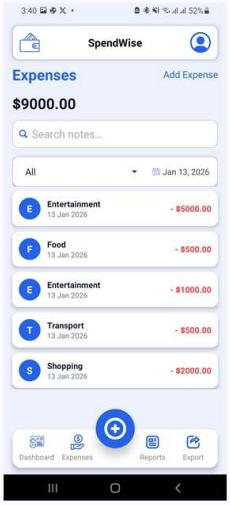
USER EXPERIENCE

Intuitive App Screens

Filter Expenses



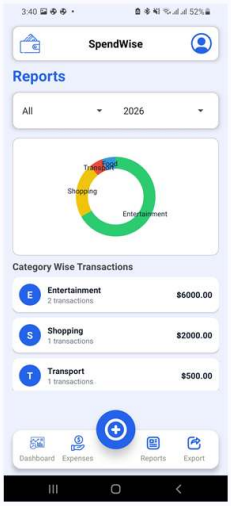


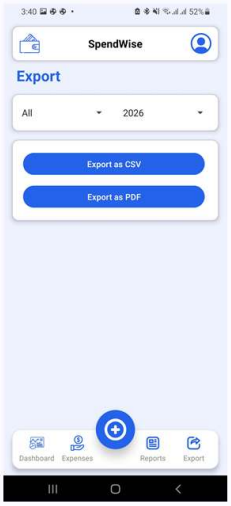


USER EXPERIENCE

Intuitive App Screens

Reports & Export

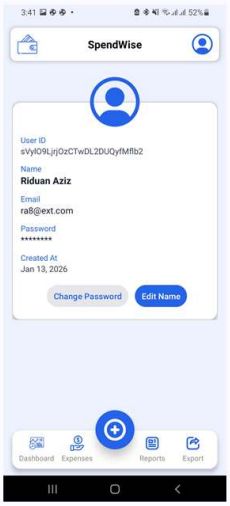


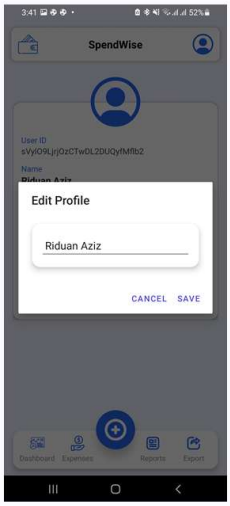


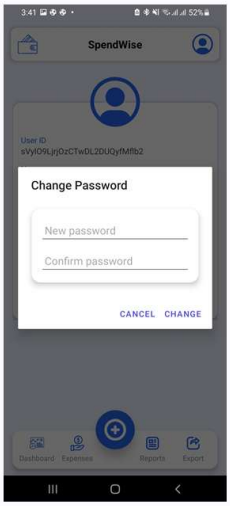
USER EXPERIENCE

Intuitive App Screens

Profile Management







4.3 Key Code Snippets

4.3.1 Activity Setup

```
public class ExpensesActivity extends AppCompatActivity {
    private RecyclerView recyclerView;
    private ExpenseAdapter adapter;
    private final List<Expense> expenseList = new
                ArrayList<>();
    private final List<Expense> allExpenses = new
                ArrayList<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_expenses);

        initViews();
        setupRecycler();
        setupFirebaseListener();
        setupFilters();
        setupBottomNavigation();
    }
}
```

4.3.2 Firebase Realtime Database Listener

```
private void setupFirebaseListener() {
    String uid =
                FirebaseAuth.getInstance().getCurrentUser().ge
                tUid();
    DatabaseReference ref = FirebaseDatabase.getInstance()

                .getReference("users").child(uid).child("expen
                ses");

    expensesQuery = ref.orderByChild("timestamp");

    expensesListener = new ValueEventListener() {
        @Override
        public void onDataChange(@NonNull DataSnapshot
                snapshot) {
            allExpenses.clear();
            for (DataSnapshot ds : snapshot.getChildren()) {
                Expense e = ds.getValue(Expense.class);
                if (e != null) {
                    e.setId(ds.getKey());
                    allExpenses.add(e);
                }
```

```
            }
            applyFiltersLocally();
        }
        @Override
        public void onCancelled(@NonNull DatabaseError error)
                {}
    };
    expensesQuery.addValueEventListener(expensesListener);
}
```

4.3.3 RecyclerView Setup

```
private void setupRecycler() {
    recyclerView.setLayoutManager(new
                LinearLayoutManager(this));
    adapter = new ExpenseAdapter(this, expenseList);

    adapter.setOnItemClickListener(expense -> {
        Intent i = new Intent(this,
                EditExpenseActivity.class);
        i.putExtra(EditExpenseActivity.EXTRA_EXPENSE_ID,
                expense.getId());
        startActivity(i);
    });

    recyclerView.setAdapter(adapter);
}
```

4.3.4 Search, Category, and Date Filters

```
private void setupFilters() {
    etSearchBox.addTextChangedListener(new TextWatcher() {
        @Override public void afterTextChanged(Editable s) {
            currentSearch = s.toString().trim();
            applyFiltersLocally();
        }
    });

    spCategoryFilter.setOnItemSelectedListener(new
                AdapterView.OnItemSelectedListener() {
        @Override
        public void onItemSelected(AdapterView<?> parent, View
                view, int pos, long id) {
            currentCategory =
                parent.getItemAtPosition(pos).toString();
            applyFiltersLocally();
        }
        @Override public void onNothingSelected(AdapterView<?>
                parent) {}
    });
```

```
    tvDateFilterView.setOnClickListener(v ->
                showDatePicker());
}
```

Chapter 5: Testing

5.1 Test Cases

Test Case ID	Input Data	Expected Result	Actual Result	Status
TC01	Valid Login	Login Success	Login Success	Pass
TC02	Add Expense	Expense Saved	Expense Saved	Pass
TC03	Edit Expense	Data Updated	Data Updated	Pass
TC04	Export CSV	File Generated	File Generated	Pass
TC05	Invalid Login	Error Message	Error Message	Pass

Chapter 6: Conclusion & Future Scope

6.1 Conclusion

SpendWise is a secure, efficient, and user-friendly expense tracking application developed using Java and Firebase. It simplifies expense management, provides valuable insights, and supports offline usage, making it suitable for daily personal finance tracking.

6.2 Future Scope

- Budget limit alerts
- Yearly analytics
- Multi-currency support
- Cloud Functions for automation
- Backup and restore feature